

Exercise Part 1 : Understanding a Specific Feature

1\. The main file related to task creation and status updates is `models.js`. It contains the `Task` class, task statuses, and functions used to update a task. The `cli.js` and `app.js` files may interact with the `Task` model when users create or update tasks. The `tests` folder is used to test that these functions work correctly.

```
2\. const TaskStatus = {  
  
  &#x20; TODO: 'todo',  
  
  &#x20; IN_PROGRESS: 'in_progress',  
  
  &#x20; REVIEW: 'review',  
  
  &#x20; DONE: 'done'  
  
};
```

```
class Task {  
  
  &#x20; constructor(title, description = '', priority = TaskPriority.MEDIUM, dueDate =  
  null, tags = []) {  
  
    &#x20; this.id = uuidv4();  
  
    &#x20; this.title = title;  
  
    &#x20; this.description = description;  
  
    &#x20; this.priority = priority;  
  
    &#x20; this.status = TaskStatus.TODO;  
  
    &#x20; this.createdAt = new Date();  
  
    &#x20; this.updatedAt = this.createdAt;
```

```
    this.dueDate = dueDate;

    this.completedAt = null;

    this.tags = tags;

  }
```

```
markAsDone() {

  this.status = TaskStatus.DONE;

  this.completedAt = new Date();

  this.updatedAt = this.completedAt;

}
```

3\ . I used the prompt template to ask the AI to explain the task creation and status update feature in the JavaScript Task Manager codebase. I included the relevant files and the models.js code snippet so that the AI could explain how tasks are created and how their status changes from TODO to DONE.

4.1 - The main component involved in task creation and updates is models.js, which contains the Task class and controls the task's information and status. cli.js handles user commands that can create or update tasks, while app.js connects the different parts of the application. The tests folder is used to check that task creation and updates work correctly.

4.2 - When a task is created, the Task class in models.js creates a new task and gives it an ID, title, description, priority, and other information. The task starts with a TODO status. When the task is marked as done, the markAsDone() method changes the status to DONE, records the completion time, and updates the last modified time.

4.3 - Task data is handled by storage.js. It is responsible for saving task information and retrieving it when the application needs it. app.js and cli.js can use the storage functions to access the saved tasks.

4.4 - One interesting pattern I noticed is that the Task class in models.js keeps the task's data and actions together. For example, the markAsDone() method changes the task status and updates the completion time. This makes the Task class responsible for managing its own information.

Exercise Part 2 :

1\ - I read through the code in models.js to understand how task priorities work. I found that there are four priority levels: LOW, MEDIUM, HIGH, and URGENT. Each priority has a number from 1 to 4. A new task has MEDIUM priority by default unless another priority is provided.

2\ - I'm exploring how task priorities work in the JavaScript Task Manager codebase.

From reading models.js, I understand that there are four task priority levels: LOW, MEDIUM, HIGH, and URGENT. They have values of 1, 2, 3, and 4 respectively. I also understand that a new task uses MEDIUM priority by default unless another priority is provided.

The main file I'm looking at is:

\- models.js.

I think the code works by using TaskPriority to define the available priority levels, and the Task constructor uses MEDIUM as the default priority when a task is created.

What's still unclear to me is how the priority value is used after a task is created and whether other files use the priority levels.

3\. - 1. The code defines four task priorities: LOW, MEDIUM, HIGH, and URGENT. Their values are 1, 2, 3, and 4.

2\. When a new task is created, its priority is set through the priority parameter in the Task constructor. If no priority is provided, MEDIUM is used as the default.

3\. The priority is stored in the task's priority property. This means each task keeps its own priority.

4\. The Task class also has an update() method that can update existing task properties, so the priority can potentially be changed after a task is created.

5\. To understand how the priority is used elsewhere, I would need to examine app.js, cli.js, and the files in the tests folder for references to TaskPriority or priority.

4.1 - Initial understanding: I thought task priorities were simply labels used to show how important a task is.

What I discovered: I found that the code defines four priority levels: LOW, MEDIUM, HIGH, and URGENT, with values from 1 to 4. I also discovered that MEDIUM is the default priority for a new task, but another priority can be provided when the task is created. The priority is stored as part of the task and can be updated later.

4.2 - The guided questions helped me understand that task priorities are defined in one place using TaskPriority. I learned that there are four levels: LOW, MEDIUM, HIGH, and URGENT. I also discovered that MEDIUM is automatically given to a new task when no priority is specified, and that the priority is stored as part of each task and can be updated.

4.3 - I initially thought that task priorities were only labels such as low, medium, and high. I discovered that each priority also has a numerical value from 1 to 4. I also thought that the priority could not be changed after creating a task, but the code shows that task properties can be updated.

Exercise Part 3 :

1\ . The entry point is the user action of marking a task as complete. The main component involved is models.js, where the markAsDone() method changes the task status to DONE and records the completion and update times.

cli.js and app.js may handle the user's command and connect it to the Task model, while storage.js may be involved in saving the updated task data. The tests folder can be used to test that completing a task works correctly.

2\ . - I found that the state change for task completion is handled mainly in app.js and models.js. In app.js, the updateTaskStatus() method checks if the new status is DONE and then calls task.markAsDone(). In models.js, markAsDone() changes the task status to DONE and updates the completion and modified dates. The updated task is then saved using storage.save().

3\ - I am working on a JavaScript Task Manager. I want to understand the complete data flow when a user marks a task as complete.

Please help me identify:

\- Where the process starts

\- Which files/components are involved

\- How the task status changes

\- How the updated information is saved

\- What happens if the task is not found

Please explain the flow in simple terms so I can understand it and document it in my assignment.

4.1

Step:	File:	What happens:
1.	cli.js	User marks task complete
2.	app.js	Receives the task ID
3.	models.js	Changes status to DONE
4.	storage.js	Saves the updated task
5.	Task Manager	Task is completed

4.2 - When a task is completed:

- 1\ The task status changes from its current status to DONE.
- 2\ The `completedAt` time is recorded.
- 3\ The `updatedAt` time is updated.
- 4\ The changes are saved.

4.3 Possible points of failure are:

- 1\ The task ID may not exist.
- 2\ The task may not be found in storage.
- 3\ The task may not be saved correctly.
- 4\ An invalid status may be entered.

4.4 - The application saves the changes to the task using `storage.js`. After the task is marked as done, `app.js` calls `this.storage.save()` to save the updated task information.

Exercise Part 4:

1. After reviewing my journal, I now understand how the Task Manager handles tasks and how the different files work together. I understand that `cli.js` receives the user's command, `app.js` handles the task operation, `models.js` manages the task status, and `storage.js` saves the changes. I also understand that when a task is completed, its status changes to DONE, the completion time is recorded, and the changes are saved.

2.1 The application is a JavaScript Task Manager that allows users to create and manage tasks.

The application is divided into different files, and each file has a specific responsibility.

\- cli.js is the user interface. It receives commands from the user, such as creating a task or changing its status.

\- app.js contains the main task management logic. It handles actions such as creating tasks, updating their status and priority, and deleting tasks.

\- models.js defines the task structure. It contains the Task class as well as the different task statuses and priorities.

\- storage.js is responsible for storing and retrieving the task data.

\- The tests folder contains tests for checking that the application works correctly.

The files work together as a flow. The user enters a command through cli.js. The request is passed to app.js, which performs the required task operation. The task information is managed using the models in models.js, and the changes are saved using storage.js.

Overall, the application uses separate components so that each part has a clear responsibility. This makes it easier to understand and maintain the Task Manager.

2.2 1. Task Creation

The user creates a task through cli.js. The information is passed to app.js, which creates a new Task using the model in models.js. The task is then saved using storage.js.

2. Task Prioritization

Each task has a priority level: Low, Medium, High, or Urgent. The priority is stored as a value from 1 to 4. The user can also update the priority of an existing task.

3. Task Completion

When the user marks a task as complete, cli.js sends the command to app.js. The task is changed to DONE in models.js. The completion time and update time are recorded, and the changes are saved.

2.3 One interesting approach I discovered is that the application separates different responsibilities into different files.

For example, cli.js handles user commands, app.js handles the main task logic, models.js manages the task structure, and storage.js handles saving the data.

I found this interesting because it keeps the application organised and makes it easier to understand and maintain.

2.4 The most challenging part for me was understanding how the different files work together when a task is completed. At first, I was not sure where the task status was changed or how the changes were saved.

The prompts helped me by guiding me to look at the important files and follow the flow step by step. This helped me understand the role of cli.js, app.js, models.js, and storage.js.